

Technical Specification v1.1 — DevOps, CI/CD & Kubernetes Implementation Specification

Absolutely. We'll continue the specification in **English**, while treating **Icelandic law, Icelandic legal practice, Icelandic language, and the Icelandic market** as first-class requirements.

The next step is therefore:

Technical Specification v1.1 — DevOps, CI/CD & Kubernetes Implementation Specification

Technical Specification v1.1

DevOps, CI/CD & Kubernetes Implementation Specification

Project: Icelandic Legal Case Management System (ILCMS)

Version: 1.1

Status: Implementation Baseline

Date: September 2026

Target Market: Iceland

Primary Language: Icelandic

Development Language: English

1. Purpose

This specification converts the infrastructure architecture defined in v1.0 into an implementable DevOps and deployment model.

The objective is to establish a reproducible, automated and secure platform on which ILCMS can be developed, tested, deployed and operated.

The specification covers:

- source-code repository structure
- containerization
- local development
- CI/CD
- Kubernetes
- Helm
- GitOps
- infrastructure as code

- secrets management
- database migrations
- observability
- backup and recovery
- security scanning
- release management

2. Design Principles

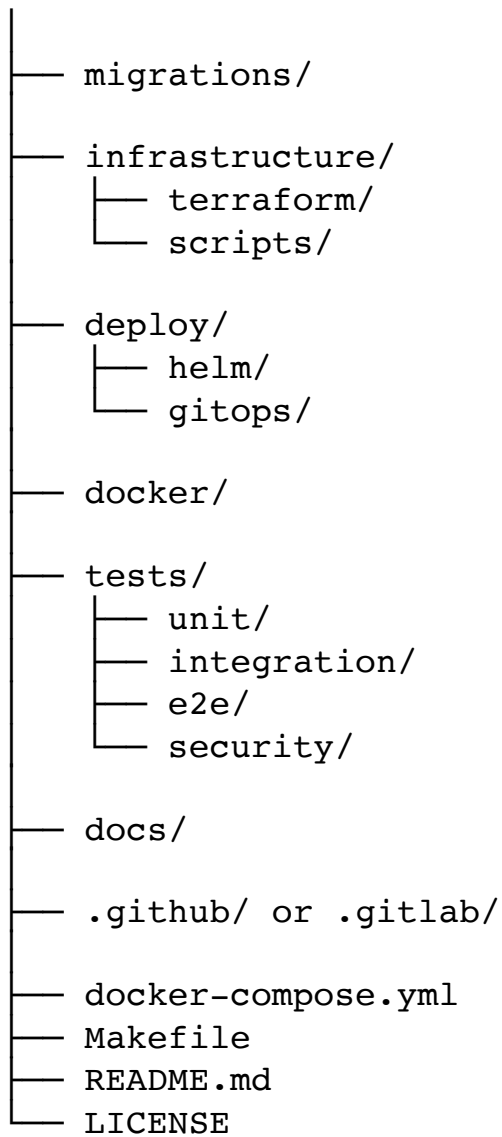
The implementation shall follow these principles:

1. **Infrastructure as Code**
2. **Configuration as Code**
3. **Git as the source of truth**
4. **Build once, deploy many**
5. **Immutable production artifacts**
6. **Automated testing**
7. **Least privilege**
8. **Zero-trust internal architecture**
9. **Auditable deployments**
10. **Automated recovery where practical**

3. Repository Architecture

The initial monorepo structure shall be:

```
ilcms/  
├── frontend/  
├── backend/  
├── worker/  
└── scheduler/
```



The exact CI provider can be changed without changing the architecture.

4. Application Components

The system shall initially contain:

frontend
backend
worker
scheduler

Supporting infrastructure:

PostgreSQL
Redis
RabbitMQ
Object Storage
Vault

Optional future components:

search
OCR
AI worker
notification service

5. Containerization

Every deployable application component shall have its own container image.

Required:

ilcms/frontend
ilcms/backend
ilcms/worker
ilcms/scheduler
Images shall be immutable.

Production shall deploy images identified by immutable version or digest.

6. Container Image Requirements

All production containers shall:

- run as non-root
- use a minimal base image
- contain only required packages
- define a health endpoint where applicable
- define explicit startup behavior
- define resource requirements
- avoid embedded credentials
- be vulnerability scanned

Example:

```
FROM minimal-runtime
```

```
USER nonroot
```

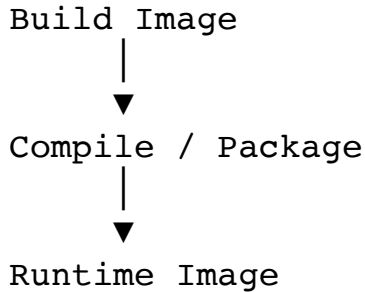
```
COPY application /
```

```
ENTRYPOINT [...]
```

7. Multi-Stage Builds

Application images should use multi-stage builds.

Example:



Development dependencies shall not be present in production images unless required.

8. Local Development

Developers shall be able to start the complete core platform using:

```
docker compose up
```

The local environment shall provide:

Frontend

Backend

Worker

Scheduler

PostgreSQL

Redis

RabbitMQ

S3-compatible object storage

A local Vault instance may be provided for development.

9. Development Configuration

Development configuration shall be externalized.

Example:

```
.env.example
```

Actual:

```
.env
```

must never be committed.

The repository shall document all required environment variables.

10. Configuration Model

Configuration shall be divided into:

Application configuration

Environment configuration

Secrets

Feature flags

Secrets shall never be treated as ordinary configuration.

11. Environment Model

Supported environments:

dev

test

staging

production

Optional:

dr

Each environment shall have independent:

- database
- object storage
- message queues
- credentials
- secrets
- Kubernetes namespace

12. Branch Strategy

Recommended:

main

```
|
|— feature/*
|— bugfix/*
```

└─ release/*

Production deployments shall originate from a protected branch or release tag.

13. Pull Request Requirements

A pull request shall require:

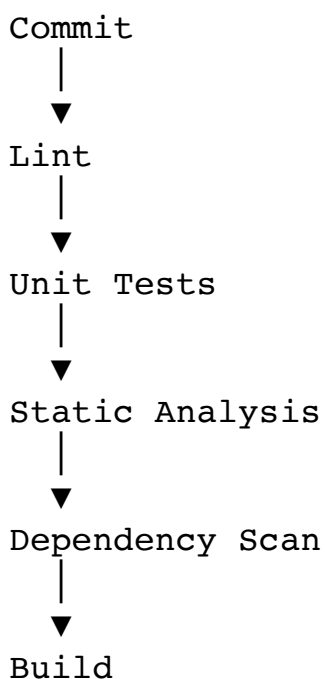
- successful build
- unit tests
- linting
- static analysis
- dependency scanning
- container build
- container vulnerability scanning

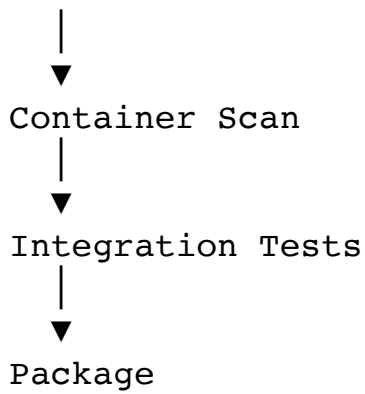
Where appropriate:

- integration tests
- E2E tests

14. CI Pipeline

Baseline pipeline:





15. Security Pipeline

Security checks shall include:

SAST

Dependency scanning

Container scanning

Secret scanning

IaC scanning

SBOM generation

Examples of tooling can be selected later.

The architecture shall remain tool-independent.

16. SBOM

Every production release shall generate a Software Bill of Materials.

The SBOM shall identify:

- application dependencies
- operating system packages
- libraries
- versions

SBOMs shall be retained with release artifacts.

17. Container Signing

Production images should be cryptographically signed.

Deployment policy should eventually require:

trusted registry
+
valid image signature
before an image can enter production.

18. Container Registry

Production images shall be stored in a private registry.

Example structure:

```
registry.example.is/  
├── ilcms/frontend  
├── ilcms/backend  
├── ilcms/worker  
└── ilcms/scheduler
```

Images shall use immutable tags.

Example:

```
backend:1.1.0  
backend:git-a83f92c  
Digest pinning is preferred for production.
```

19. Kubernetes Architecture

Production Kubernetes shall contain:

Ingress
Frontend
Backend
Worker
Scheduler

Supporting infrastructure may either be deployed into Kubernetes or provided as managed/external services.

20. Namespace Structure

Production:

ilcms-prod

Staging:

ilcms-staging

Test:

`ilcms-test`

Development:

`ilcms-dev`

Infrastructure:

`monitoring`

`logging`

`security`

`ingress-system`

21. Kubernetes Labels

All ILCMS resources shall use standardized labels.

Example:

```
app.kubernetes.io/name: ilcms
```

```
app.kubernetes.io/component: backend
```

```
app.kubernetes.io/version: "1.1.0"
```

```
app.kubernetes.io/managed-by: helm
```

Labels shall support operational discovery and monitoring.

22. Helm

The ILCMS application shall be packaged as a Helm chart.

Example:

```
deploy/helm/ilcms/
```

```
├── Chart.yaml
```

```
├── values.yaml
```

```
├── values-dev.yaml
```

```
├── values-test.yaml
```

```
├── values-staging.yaml
```

```
├── values-production.yaml
```

```
└── templates/
```

23. Helm Values

Environment-specific configuration shall be supplied through values.

Example:

```
replicaCount:
  api: 3
  worker: 2
```

```
resources:
  api:
    requests:
      cpu: "250m"
      memory: "512Mi"
```

Secrets shall not be stored in plain Helm values.

24. Kubernetes Deployments

Backend:

```
Deployment
Service
HorizontalPodAutoscaler
PodDisruptionBudget
NetworkPolicy
Frontend:
```

```
Deployment
Service
NetworkPolicy
Workers:
```

```
Deployment
HPA where appropriate
NetworkPolicy
Scheduler:
```

```
Deployment
or Kubernetes-native scheduling where appropriate.
```

25. Replica Requirements

Production baseline:

```
Frontend: 2+
Backend: 2+
Worker: 2+
```

The actual number shall be determined by load testing.

26. Pod Anti-Affinity

Critical replicas should be distributed across nodes.

Example concept:

API Pod 1 → Node A

API Pod 2 → Node B

API Pod 3 → Node C

This prevents one node failure from taking down the entire service.

27. Pod Disruption Budgets

Critical workloads shall define PodDisruptionBudgets.

Example:

`minimumAvailable: 1`
for small deployments.

Production values shall reflect the replica count.

28. Resource Requests

Every production workload shall define CPU and memory requests.

Example:

```
resources:
  requests:
    cpu: 250m
    memory: 512Mi
```

29. Resource Limits

Every production workload shall define appropriate limits.

Limits shall be determined through performance testing rather than arbitrary values.

30. Health Checks

Every service shall provide appropriate health checks.

Backend:

/health

/readiness

Kubernetes:

livenessProbe

readinessProbe

startupProbe

31. Ingress

Ingress shall provide:

- HTTPS
- host-based routing
- TLS certificates
- security headers
- rate limiting where appropriate

Example:

https://app.example.is



Frontend

https://api.example.is



Backend

32. TLS Certificates

Certificate management shall be automated.

Preferred architecture:

cert-manager



Certificate Authority



Kubernetes Secret

Certificates shall renew automatically.

33. Network Policies

Default Kubernetes network policy:

DENY ALL

Required connections are explicitly permitted.

Example:

Frontend → Backend

Backend → PostgreSQL

Backend → Redis

Backend → RabbitMQ

Worker → RabbitMQ

Worker → Object Storage

34. PostgreSQL Deployment

PostgreSQL shall be treated as a critical stateful service.

Preferred production options:

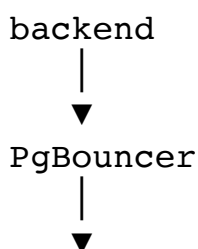
1. managed PostgreSQL
2. CloudNativePG
3. Patroni-based PostgreSQL

The application shall not depend on a specific implementation.

35. PostgreSQL Connectivity

Backend connects through a controlled database endpoint.

Example:



PostgreSQL

Connection pooling shall be enabled where required.

36. Database Migration Architecture

Migrations shall be version controlled.

Example:

```
migrations/  
├── 001_initial.sql  
├── 002_cases.sql  
├── 003_documents.sql  
└── ...
```

Migration execution shall be automated.

37. Migration Rules

Migrations shall be:

- repeatable
- versioned
- tested
- backward compatible where possible
- auditable

Production schema changes shall not normally be executed manually.

38. Expand/Contract Pattern

Breaking database changes shall follow:

```
Expand  
↓  
Deploy compatible application  
↓  
Migrate data  
↓  
Switch application  
↓
```

Contract

This supports zero/minimal downtime deployments.

39. RabbitMQ

RabbitMQ shall be deployed as a highly available messaging system where required.

Queues include:

```
document.processing
document.scanning
document.indexing
notifications
ai.processing
audit.events
```

40. Retry Policy

Asynchronous jobs shall support controlled retries.

Example:

Attempt 1

↓

Failure

↓

Retry

↓

Failure

↓

Retry

↓

Dead Letter Queue

Retry count and delay shall be configurable.

41. Dead Letter Queues

Failed messages shall not disappear silently.

Each critical queue shall have a DLQ.

Operations shall be able to inspect and replay appropriate messages.

42. Redis

Redis shall be used for non-authoritative transient state.

Permitted:

cache
rate limits
distributed locks
temporary processing state

Not permitted:

legal case records
authoritative document metadata
audit records

43. Object Storage

Production object storage shall be S3-compatible.

Required capabilities:

versioning
encryption
access control
audit logging
retention
optional immutability

44. Storage Bucket Separation

Logical buckets:

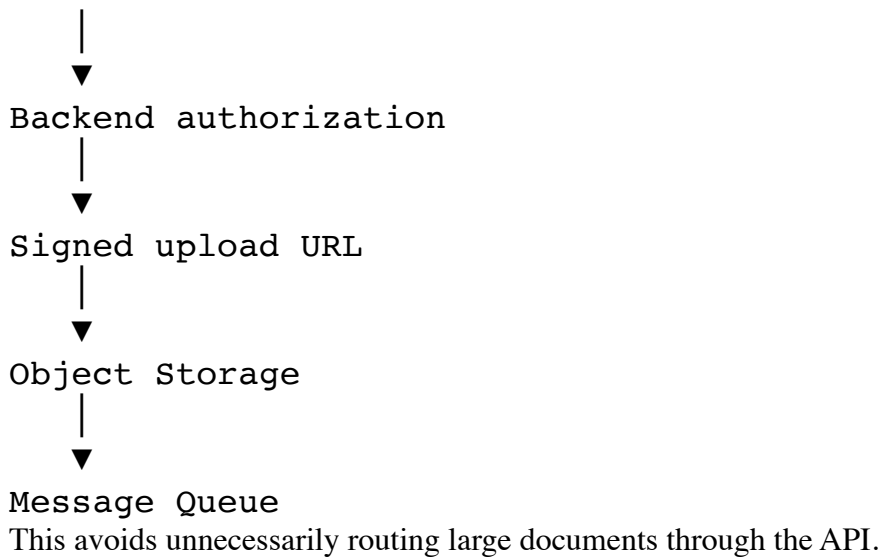
ilcms-documents
ilcms-evidence
ilcms-exports
ilcms-backups

Access shall be restricted independently.

45. Document Upload

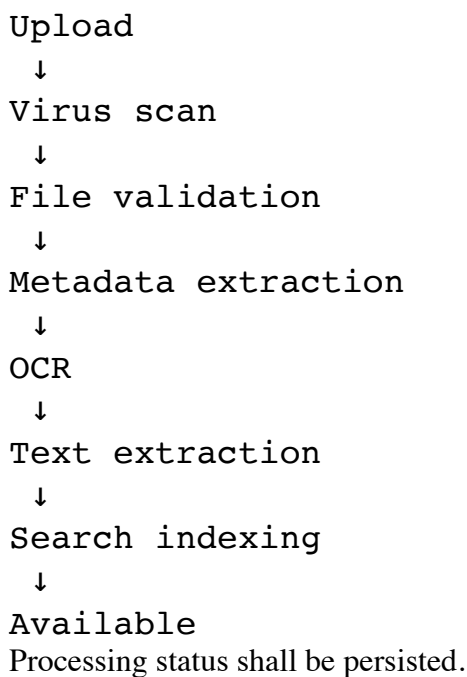
Upload flow:

Browser



46. Document Processing Worker

Worker pipeline:



47. Vault

Secrets shall be stored in HashiCorp Vault or equivalent.

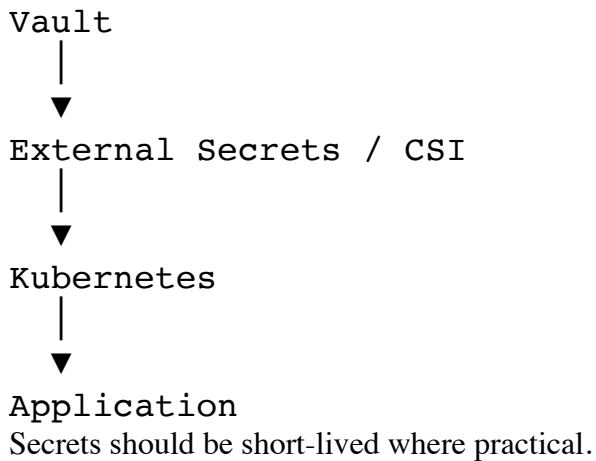
Examples:

database credentials
storage credentials
OAuth secrets

AI provider credentials
signing keys
integration credentials

48. Vault Integration

Preferred model:



49. Secret Rotation

The architecture shall support secret rotation without rebuilding application images.

Examples:

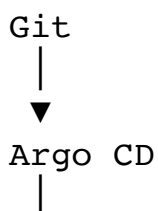
database credentials
API keys
TLS certificates
OAuth credentials

50. GitOps

Production Kubernetes state shall be controlled through GitOps.

Recommended:

Argo CD
Architecture:





Kubernetes

51. GitOps Repository

Example:

```
deploy/gitops/
├── environments/
│   ├── dev/
│   ├── test/
│   ├── staging/
│   └── production/
└── applications/
```

52. Deployment Promotion

Release flow:

```
Build image
  ↓
Test
  ↓
Push registry
  ↓
Update staging
  ↓
Automated validation
  ↓
Approval
  ↓
Update production
```

53. Build Once, Deploy Many

The same immutable application image shall be promoted between environments.

Example:

`backend@sha256:ABC`
is tested in staging and subsequently deployed to production.

Production shall not rebuild the image.

54. Rollback

Application rollback:

Current

↓

Version 1.2.0

↓

Failure

↓

Version 1.1.5

GitOps should make rollback a Git revision change.

55. Database Rollback

Database rollback shall not depend on blindly reversing migrations.

Preferred strategy:

forward-compatible migration

+

application rollback compatibility

Critical destructive migrations require additional review.

56. CI/CD Security

CI/CD credentials shall:

- use short-lived credentials where possible
- have minimum required permissions
- not be embedded in repository files
- be audited
- be rotated

57. Production Deployment Approval

Production deployments shall require explicit authorization unless an approved automated deployment policy is established.

The deployment record shall include:

```
version
Git commit
operator/automation identity
timestamp
result
```

58. Infrastructure as Code

Terraform shall define infrastructure where applicable.

Example:

```
infrastructure/terraform/
├── modules/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── production/
└── main.tf
```

59. Terraform State

Terraform state shall be stored remotely.

Requirements:

```
encrypted
versioned
access controlled
backed up
```

State shall never be committed to Git.

60. Infrastructure Drift

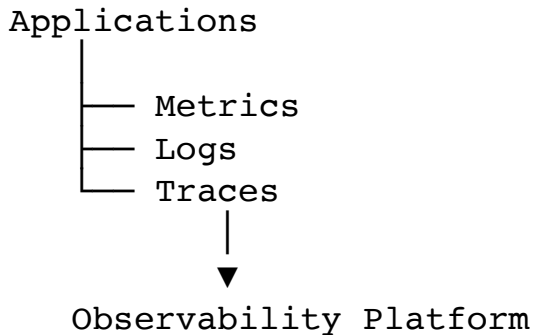
Production infrastructure shall be periodically checked for drift.

Unexpected manual changes shall be investigated.

61. Observability Stack

Baseline:

Prometheus
Grafana
OpenTelemetry
Loki / equivalent
Architecture:



62. Application Metrics

Backend shall expose metrics including:

HTTP requests
request latency
HTTP errors
database latency
queue operations
document processing
authentication failures

63. Business Metrics

Operational monitoring shall distinguish technical metrics from business metrics.

Examples:

open cases
documents processed
overdue tasks
pending workflows
failed document processing jobs
Sensitive legal content shall not be included unnecessarily.

64. Logging

Structured JSON logging is recommended.

Example:

```
{  
  "timestamp": "...",  
  "level": "INFO",  
  "service": "backend",  
  "request_id": "...",  
  "event": "document.upload.completed"  
}
```

65. Privacy-Aware Logging

Logs shall not contain unnecessary:

- case contents
- document contents
- passwords
- authentication tokens
- secret values

PII should be minimized.

66. Audit Logging

Legal-system audit events shall be immutable or tamper-resistant.

Important events include:

- LOGIN
- LOGOUT
- CASE_CREATED
- CASE_VIEWED
- DOCUMENT_UPLOADED
- DOCUMENT_VIEWED
- DOCUMENT_DOWNLOADED
- DOCUMENT_DELETED
- PERMISSION_CHANGED
- CASE_EXPORTED
- LEGAL_HOLD_CREATED
- LEGAL_HOLD_RELEASED

67. Audit Storage

Audit records shall be stored independently enough from ordinary application logs that operational log rotation cannot accidentally remove legal audit history.

Retention shall be policy-driven.

68. Backup Automation

Backup jobs shall be automated.

PostgreSQL:

full backup
+
WAL archiving
Object storage:

versioning
+
replication/backup

69. Backup Monitoring

A backup job is considered successful only if:

backup completed
+
backup verified
Failed backups shall trigger alerts.

70. Restore Automation

A separate restore procedure shall be maintained.

Example:

backup
↓
restore isolated environment
↓
database integrity checks
↓
application validation
↓
report

71. Disaster Recovery Environment

The DR environment shall be deployable from:

Terraform

+

GitOps

+

backup data

The goal is to avoid relying on undocumented manual configuration.

72. Icelandic Market Requirements

Although the implementation language is English, the product architecture shall explicitly support Icelandic legal users.

This includes:

- Icelandic language
- Icelandic names
- Icelandic addresses
- Icelandic dates
- Icelandic legal terminology
- Icelandic court terminology
- Icelandic document formats
- Icelandic time zone
- Icelandic regulatory requirements

73. Localization

The application shall use internationalization from the beginning.

Example:

en

is

English may initially be used for development/admin interfaces.

The production user interface shall prioritize Icelandic.

74. Icelandic Character Support

All system components shall use UTF-8.

The system must correctly support:

Á
á
Ð
ð
É
é
Í
í
Ó
ó
Ú
ú
Ý
ý
Þ
þ
Æ
æ
Ö
ö

Database collation and search behavior shall be tested specifically for Icelandic.

75. Icelandic Names

The data model must not assume that users have:

`first_name`

`last_name`

as a universally sufficient model.

Legal parties may have:

- personal names
- patronymic/matronymic names
- company names
- organizational names

- foreign names

Search and display logic must preserve the authoritative legal name.

76. Date and Time

Canonical storage:

UTC

User presentation:

Europe/Reykjavik

The system must correctly handle Iceland's daylight-saving rules—or lack thereof—rather than assuming a generic European timezone.

77. Icelandic Legal Documents

The document architecture must support common formats including:

PDF

PDF/A where applicable

DOCX

XLSX

PPTX

TXT

images

email exports

scanned documents

Exact accepted formats shall be configurable.

78. Icelandic Legal Terminology

Legal terminology shall not be embedded directly throughout frontend code.

Instead:

translation keys

+

terminology dictionary

shall be used.

Example:

case

document
party
deadline
court
filing
legal hold

can have localized Icelandic terminology without changing application logic.

79. Search and Icelandic Language

Search shall eventually support Icelandic linguistic characteristics.

Potential requirements include:

Unicode normalization
accent handling
stemming
tokenization
Icelandic-specific linguistic processing
These requirements shall be validated using real Icelandic legal text.

80. Data Residency

For Icelandic legal customers, deployment shall support infrastructure where customers require data to remain within a defined geographic jurisdiction.

The architecture therefore supports:

Iceland-hosted deployment
EU-hosted deployment
customer-controlled/on-premises deployment
private cloud

The actual permitted deployment location shall be determined by customer and legal requirements.

81. Customer Isolation

If the system becomes multi-tenant, tenant isolation shall exist at application and data layers.

Possible models:

shared database + tenant_id
separate schema
separate database

separate cluster

For high-security customers, dedicated database or deployment isolation may be offered.

82. Deployment Models

ILCMS should support three deployment tiers.

Tier 1 — SaaS

Customer



ILCMS hosted platform

Tier 2 — Private Cloud

Customer



Dedicated ILCMS environment

Tier 3 — On-Premises

Customer infrastructure



Kubernetes



ILCMS

This is particularly important for organizations with strict confidentiality requirements.

83. Air-Gapped Deployment

The architecture should permit a future air-gapped deployment.

This requires:

private container registry

offline package repository

local AI capability

offline documentation

offline license mechanism

local monitoring

local update process

No production dependency should inherently require Internet connectivity unless explicitly identified.

84. AI Deployment

AI must remain an optional subsystem.

Possible configurations:

No AI

Local LLM

Private LLM

Approved external AI provider

Core case-management functionality must operate without AI.

85. AI Security Boundary

AI services shall receive only the minimum data required.

Example:

Case

↓

Authorization

↓

AI data preparation

↓

Redaction/filtering if required

↓

AI service

↓

Result validation

↓

Audit

AI output shall never automatically become authoritative legal fact.

86. AI Release Architecture

AI models shall be independently versioned.

Example:

application: 1.1.0

model: legal-assistant-0.4

prompt-set: 12

This permits reproducibility and auditing.

87. Release Versioning

ILCMS shall use Semantic Versioning:

MAJOR.MINOR.PATCH

Example:

1.0.0

1.1.0

1.1.1

Security releases shall be clearly identified.

88. Release Artifact

Each release shall contain:

Git commit

container digest

SBOM

database migration version

Helm chart version

release notes

security scan results

89. Production Readiness Gate

A release may enter production only when:

- CI successful
- security scans passed
- image signed
- SBOM generated
- integration tests passed
- migration tested
- staging deployment successful
- backup verified
- rollback procedure verified

- release approved

90. Operational Runbooks

The repository shall contain operational documentation:

```
docs/runbooks/  
├── deployment.md  
├── rollback.md  
├── database-recovery.md  
├── disaster-recovery.md  
├── certificate-renewal.md  
├── secret-rotation.md  
├── incident-response.md  
└── troubleshooting.md
```

91. Incident Management

Production incidents shall follow:

```
Detect  
↓  
Classify  
↓  
Contain  
↓  
Investigate  
↓  
Recover  
↓  
Validate  
↓  
Document  
↓  
Post-incident review
```

92. Security Incident

Security incidents shall have a separate procedure covering:

```
credential compromise  
unauthorized access  
data exposure
```

malware

suspicious downloads

privilege escalation

The procedure shall include preservation of relevant evidence and audit records.

93. Developer Access

Developers shall normally have:

local development access

development cluster access

test access

Production access shall be exceptional and controlled.

94. Production Operator Access

Production operators require:

MFA

individual accounts

RBAC

audit logging

time-limited privilege where possible

Shared administrator accounts are prohibited.

95. Break-Glass Procedure

Emergency access shall be available.

Break-glass events shall record:

person

time

reason

actions

duration

result

and require post-event review.

96. Dependency Updates

Automated dependency update tooling should create pull requests.

Updates must pass:

tests
security scans
compatibility checks
before merging.

97. Vulnerability Management

Vulnerabilities shall be classified according to severity.

Critical vulnerabilities affecting exposed production components receive highest priority.

The project shall maintain a documented remediation SLA.

98. Infrastructure Testing

Infrastructure code shall be tested before production deployment.

Testing may include:

Terraform validation
Helm lint
Kubernetes manifest validation
policy validation
security scanning

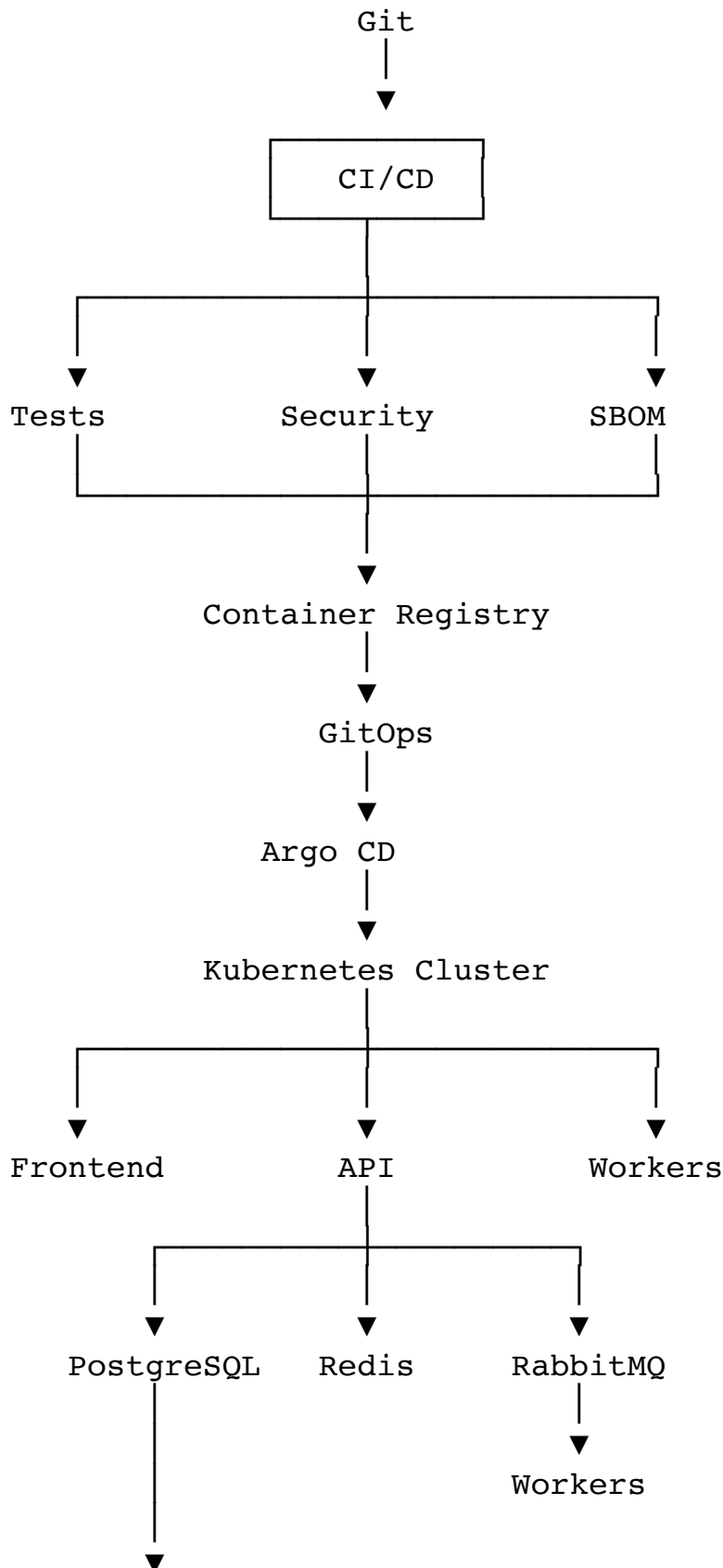
99. End-to-End Test Environment

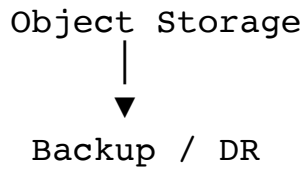
E2E tests shall validate critical user journeys.

Initial journeys:

Login
Create case
Add party
Upload document
View document
Create task
Create deadline
Search case
Export case
Audit event generated

100. Deployment Architecture — Final Baseline





101. Implementation Priorities

Implementation should proceed in this order.

Phase 1 — Developer Platform

Git repository
Docker
Docker Compose
PostgreSQL
Redis
RabbitMQ
Object Storage

Phase 2 — Application

Frontend
Backend
Worker
Database migrations

Phase 3 — CI

Tests
Security scanning
Container builds
Registry
SBOM

Phase 4 — Kubernetes

Cluster
Namespaces
Helm
Ingress
TLS
NetworkPolicies

Phase 5 — GitOps

Argo CD
Staging

Production

Rollback

Phase 6 — Operations

Monitoring

Logging

Alerting

Backup

Restore

DR

102. Definition of Done — v1.1

v1.1 is considered implemented when:

- repository structure exists
- local Docker environment works
- frontend container builds
- backend container builds
- worker container builds
- scheduler container builds
- PostgreSQL runs
- Redis runs
- RabbitMQ runs
- object storage runs
- database migrations work
- CI pipeline works
- security scanning works
- SBOM is generated
- container registry is operational
- Helm chart exists
- Kubernetes deployment works
- TLS works

- NetworkPolicies work
- Vault integration works
- GitOps deployment works
- staging deployment works
- production deployment process is documented
- rollback works
- backups work
- restore has been tested
- monitoring works
- centralized logging works
- audit logging works
- Icelandic localization framework exists
- Icelandic UTF-8/search requirements have been tested

103. Architecture Status

With v1.1, the ILCMS specification has progressed from architectural design toward an executable engineering blueprint.

Requirements



Data Model



Database & API



Workflow & Security

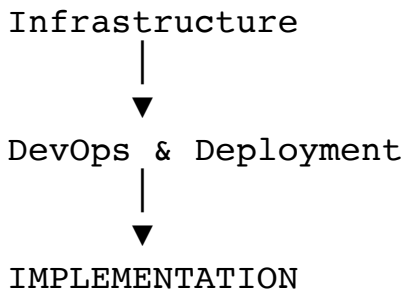


Application Architecture



Frontend / UX





104. Next Specification

The logical next step is:

Technical Specification v1.2 — Backend Implementation Specification

This should define the actual backend engineering contract, including:

- Python project structure
- FastAPI application architecture
- domain modules
- service layer
- repository layer
- dependency injection
- authentication
- authorization
- RBAC/ABAC
- PostgreSQL access
- SQLAlchemy models
- Pydantic schemas
- API versioning
- error handling
- audit events
- document API
- workflow API

- task/deadline API
- search API
- background jobs
- event architecture
- testing strategy
- OpenAPI conventions
- Icelandic localization interfaces

At that point we can move from **"how do we deploy ILCMS?"** to **"exactly how do we implement the ILCMS backend?"**.

The v1.2 specification can also establish the concrete Python/FastAPI code structure so that the first actual application skeleton can be generated directly from the specification.